Manejo de Datos en R

R Base vs Tidyverse vs data.table

Especialización en Ciencia de Datos
Universidad Nacional del Oeste (UNO)
2025

Información del Curso

Materia: Aplicaciones de Ciencia de Datos (07102)

Institución: Universidad Nacional del Oeste (UNO)

Carrera: Especialización en Ciencia de Datos

Año: 1er año

Modalidad: Presencial - Trimestral

Agenda

1. Introducción: Tres paradigmas de manipulación de datos en R
2. R Base: El enfoque tradicional y sus fundamentos
3. Tidyverse: Código legible y filosofía del 'tidy data'
4. data.table: Optimización para alto rendimiento
5. Comparación sintáctica: Operaciones comunes en los tres enfoques
6. Benchmark: Análisis de rendimiento con datasets reales
7. Recomendaciones prácticas para proyectos de ciencia de datos

Tres Paradigmas de Manipulación de Datos

- R Base: El punto de partida
- - Incluido en toda instalación estándar de R
- - Sintaxis tradicional utilizando operadores $ y [ ]
- - No requiere instalación de paquetes adicionales
- - Base sólida para entender R en profundidad

- Tidyverse: El enfoque moderno
- - Colección integrada de paquetes para ciencia de datos
- - Sintaxis consistente y altamente legible (dplyr, tidyr, readr)
- - Filosofía del 'pipe' (%>% o |>) para operaciones encadenadas
- - Ideal para análisis exploratorio y comunicación de resultados

- data.table: La solución de alto rendimiento
- - Extensión optimizada del data.frame de R base
- - Sintaxis concisa y poderosa: DT[i, j, by]
- - Diseñado específicamente para procesamiento de grandes volúmenes
- - Modificación por referencia para eficiencia de memoria

R Base: El Enfoque Tradicional

✓ Ventajas principales:

- Disponibilidad inmediata: no requiere instalación de paquetes
- Estabilidad garantizada: parte del núcleo de R
- Documentación exhaustiva y comunidad establecida
- Funciona en cualquier entorno R sin dependencias
- Ideal para scripts simples y análisis rápidos

✗ Limitaciones a considerar:

- Sintaxis menos consistente entre diferentes funciones
- Puede resultar verboso en operaciones complejas
- Menor rendimiento con datasets grandes (>100K filas)
- Código menos intuitivo para análisis multi-paso
- Curva de aprendizaje empinada para principiantes

R Base: Importación de Datos

La función read.csv() es la herramienta estándar para leer archivos CSV. Es importante especificar stringsAsFactors = FALSE para evitar conversiones automáticas no deseadas.

Lectura básica de archivo CSV:

```
df <- read.csv("datos.csv", stringsAsFactors = FALSE)
```

→ *Lee el archivo y crea un data.frame. El parámetro stringsAsFactors evita que el texto se convierta a factores.*

Lectura de archivo delimitado por tabulaciones:

```
df <- read.delim("datos.txt", sep = "\t", header = TRUE)
```

→ *Útil para archivos .txt o .tsv. El parámetro sep especifica el delimitador.*

Control de encoding y tipos de datos:

```
df <- read.csv("datos.csv", encoding = "UTF-8",
        na.strings = c("NA", ""), dec = ",")
```

→ *Permite especificar encoding para caracteres especiales y definir valores perdidos.*

R Base: Selección y Filtrado

La indexación con corchetes [ ] es el mecanismo fundamental de R base. La estructura es: df[filas, columnas]

Seleccionar columnas específicas:

```
df_subset <- df[, c("nombre", "edad", "ciudad")]
```

→ *El operador [, c(...)] selecciona columnas por nombre. Las filas vacías indican "todas las filas".*

Filtrar filas con condiciones lógicas:

```
df_filtrado <- df[df$edad > 30 & df$ciudad == "Buenos Aires", ]
```

→ *Usa operadores lógicos (&, |, ==) para filtrar. El $ accede a columnas individuales.*

Combinación de selección y filtrado:

```
resultado <- df[df$edad > 25, c("nombre", "salario")]
```

→ *Aplica filtro en filas Y selección en columnas simultáneamente.*

R Base: Creación y Modificación de Columnas

El operador $ permite acceder y crear columnas. Cada asignación crea o modifica una columna del data.frame.

Crear nueva columna por cálculo:

```
df$ganancia <- df$ingresos - df$costos
```

→ *Operaciones aritméticas elemento por elemento. La columna se crea automáticamente.*

Transformación condicional con ifelse:

```
df$categoria <- ifelse(df$edad < 30, "Joven",
                ifelse(df$edad < 60, "Adulto", "Mayor"))
```

→ *ifelse() evalúa condiciones y asigna valores. Se pueden anidar para múltiples categorías.*

Aplicar funciones a columnas:

```
df$nombre_upper <- toupper(df$nombre)
df$edad_normalizada <- scale(df$edad)
```

→ *Funciones como toupper(), scale(), log() se aplican elemento por elemento.*

R Base: Agregaciones y Resúmenes por Grupo

aggregate() es la función estándar para cálculos por grupo. La sintaxis fórmula (~) separa variables de agrupación.

Promedio por categoría con aggregate:

```
resultado <- aggregate(ventas ~ categoria, data = df, FUN = mean)
```

→ *Sintaxis: variable_a_resumir ~ variable_agrupadora. FUN especifica la función (mean, sum, etc.).*

Múltiples variables de agrupación:

```
resultado <- aggregate(ventas ~ region + año, data = df, FUN = sum)
```

→ *Usa + para agrupar por múltiples variables. Calcula sum de ventas por cada combinación.*

Alternativa con tapply (más directo):

```
promedios <- tapply(df$ventas, df$categoria, mean)
```

→ *tapply() devuelve un vector/matriz en lugar de data.frame. Útil para cálculos rápidos.*

Tidyverse: Código Legible y Expresivo

✓ Ventajas principales:

- Sintaxis consistente y predecible en todos los paquetes
- Filosofía del 'pipe' (|>) para código lineal y legible
- Excelente para análisis exploratorio y visualización
- Comunidad activa y documentación ejemplar
- Integración perfecta con ggplot2 para gráficos
- Manejo inteligente de tipos de datos (tibbles)

✗ Consideraciones importantes:

- Requiere instalación de múltiples paquetes
- Puede ser más lento que data.table con datasets muy grandes
- Mayor uso de memoria en operaciones complejas
- Abstracción puede dificultar debugging en casos

Tidyverse: Ecosistema de Paquetes

El Tidyverse es una colección integrada de 8 paquetes principales:

- readr: Importación rápida y consistente de datos
  - read_csv(), read_delim(), read_excel()

- dplyr: Manipulación de datos (el corazón del Tidyverse)
  - filter(), select(), mutate(), summarise(), arrange()

- tidyr: Reestructuración de datos (tidy data)
  - pivot_longer(), pivot_wider(), separate(), unite()

- ggplot2: Visualización basada en gramática de gráficos

- stringr, forcats, lubridate, purrr: Herramientas especializadas

Instalación: install.packages('tidyverse')

Tidyverse: Importación con readr

readr ofrece funciones de importación optimizadas que detectan tipos de datos inteligentemente y son ~10x más rápidas que R base.

Lectura optimizada de CSV:

```
library(readr)
df <- read_csv("datos.csv")
```

→ *Detecta automáticamente tipos de columnas. Devuelve un tibble (versión mejorada de data.frame).*

Especificar tipos de columnas:

```
df <- read_csv("datos.csv",
        col_types = cols(edad = col_integer(),
                         fecha = col_date(format = "%Y-%m-%d")))
```

→ *col_types permite control explícito sobre tipos de datos, evitando conversiones incorrectas.*

Leer desde URL directamente:

```
df <- read_csv("https://ejemplo.com/datos.csv")
```

→ *readr puede importar directamente desde internet sin descarga previa.*

Tidyverse: Operaciones Básicas con dplyr

dplyr usa verbos intuitivos que describen la acción. Cada función hace una cosa y la hace bien.

Seleccionar columnas con select():

```
df |> select(nombre, edad, ciudad)
```

→ select() elige columnas por nombre. No requiere comillas. El pipe |> pasa el resultado.

Filtrar filas con filter():

```
df |> filter(edad > 30, ciudad == "Buenos Aires")
```

→ filter() usa condiciones lógicas separadas por comas (equivalente a &).

Crear/modificar columnas con mutate():

```
df |> mutate(ganancia = ingresos - costos,
      edad_meses = edad * 12)
```

→ mutate() crea múltiples columnas en una sola llamada. Puede usar columnas recién creadas.

Tidyverse: El Poder del Pipe Operator

El pipe (|>) permite encadenar operaciones, creando código que se lee como una receta: 'toma los datos, luego haz X, luego Y, luego Z'.

Análisis multi-paso con pipes:

```
resultado <- df |>
  filter(fecha >= "2024-01-01") |>
  mutate(trimestre = quarter(fecha)) |>
  group_by(region, trimestre) |>
  summarise(total_ventas = sum(ventas)) |>
  arrange(desc(total_ventas))
```

→ Cada línea representa una operación clara. Se lee de arriba hacia abajo, sin anidamiento.

Equivalente sin pipes (menos legible):

```
resultado <- arrange(
  summarise(group_by(mutate(filter(df,
    fecha >= "2024-01-01"), trimestre =
quarter(fecha)),
    region, trimestre), total_ventas =
sum(ventas)),
  desc(total_ventas))
```

→ Sin pipes: hay que leer de adentro hacia afuera. Difícil de entender y mantener.

Tidyverse: Agregaciones con group_by() y summarise()

La combinación group_by() + summarise() es el patrón estándar para análisis por grupos en tidyverse.

Resumen por un grupo:

```
df |>
  group_by(categoria) |>
  summarise(
    promedio = mean(ventas),
    total = sum(ventas),
    cantidad = n()
  )
```

→ group_by() define grupos. summarise() aplica funciones a cada grupo. n() cuenta filas.

Múltiples grupos y estadísticas:

```
df |>
  group_by(region, año) |>
  summarise(
    across(c(ventas, costos),
           list(promedio = mean, total =
sum))
  )
```

→ across() aplica múltiples funciones a múltiples columnas eficientemente.

data.table: Velocidad y Eficiencia Extrema

✓ Ventajas principales:

- Rendimiento excepcional: 10-100x más rápido que R base
- Sintaxis concisa y poderosa: DT[i, j, by]
- Modificación por referencia: ahorra memoria dramáticamente
- fread(): lectura ultra-rápida de archivos (5-20x más rápido)
- Ideal para producción y procesamiento de big data
- Join optimizado y operaciones complejas eficientes

✗ Consideraciones importantes:

- Curva de aprendizaje más pronunciada inicialmente
- Sintaxis menos intuitiva para principiantes
- Documentación técnica (requiere más estudio)
- Menor énfasis en legibilidad vs. rendimiento

data.table: La Sintaxis DT[i, j, by]

La estructura DT[i, j, by] es la base de data.table. Cada componente tiene un propósito específico y se puede usar independientemente:

i: Filas (WHERE)

Qué filas seleccionar o filtrar

Equivalente a WHERE en SQL

Ejemplo: DT[edad > 30]
DT[ciudad == "Buenos Aires"]

j: Columnas (SELECT)

Qué hacer con las columnas

Seleccionar, crear, computar

Ejemplo: DT[, .(nombre, edad)]
DT[, sum(ventas)]

by: Agrupar (GROUP BY)

Por qué variable(s) agrupar

Operaciones por grupo

Ejemplo: DT[, sum(ventas), by = region]
DT[, .N, by = .(año, mes)]

data.table: Importación Ultra-Rápida con fread()

fread() (Fast Read) es una de las funciones más optimizadas de R. Detecta automáticamente formato y tipos de datos.

Lectura básica (automática):

```
library(data.table)
DT <- fread("datos.csv")
```

→ *Detecta automáticamente separadores, tipos de columnas, y encoding. 5-20x más rápido que read.csv().*

Importar solo columnas específicas:

```
DT <- fread("datos.csv", select = c("nombre", "edad", "ventas"))
```

→ *select permite leer solo columnas necesarias, ahorrando tiempo y memoria en archivos grandes.*

Leer directamente desde URL o comando:

```
DT <- fread("https://ejemplo.com/datos.csv")
DT <- fread("curl https://api.ejemplo.com/data")
```

→ *fread puede leer desde internet o ejecutar comandos shell para obtener datos.*

data.table: Operaciones Básicas

data.table usa su sintaxis especial dentro de [ ]. El operador := permite modificación por referencia (sin copiar).

Seleccionar columnas con .():

```
DT[, .(nombre, edad, ciudad)]
```

→ *.() es un atajo para list(). Selecciona columnas y devuelve un data.table.*

Filtrar filas (sin comas):

```
DT[edad > 30 & ciudad == "Buenos Aires"]
```

→ *En el argumento i (filas), no se necesita $ para referirse a columnas. Sin coma final.*

Crear columnas por referencia con :=

```
DT[, ganancia := ingresos - costos]
DT[, c("total", "promedio") := .(sum(ventas), mean(ventas))]
```

→ *:= modifica el data.table en su lugar, sin copiar. Extremadamente eficiente en memoria.*

data.table: Agregaciones Eficientes

El patrón DT[i, j, by] permite agregaciones complejas en una sola línea. .N es un símbolo especial que cuenta filas.

Resumen por grupo simple:

```
DT[, .(promedio = mean(ventas),
       total = sum(ventas),
       cantidad = .N),
   by = categoria]
```

→ *.N cuenta el número de filas en cada grupo. Equivalente a n() en dplyr.*

Múltiples grupos y filtro previo:

```
DT[fecha >= "2024-01-01",
   .(ventas_totales = sum(ventas)),
   by = .(region, trimestre = quarter(fecha))]
```

→ *Filtra en i, calcula en j, agrupa en by. Todo en una expresión eficiente.*

Agregar y ordenar resultado:

```
DT[, .N, by = ciudad][order(-N)]
```

→ *Encadenar operaciones: primer [ ] agrupa y cuenta, segundo [ ] ordena descendente.*

Comparación Lado a Lado: Filtrar y Resumir

Tarea: Filtrar personas mayores de 30 años y calcular el promedio de ventas por categoría

R Base:

```
# Primero filtrar, luego agregar
resultado <- aggregate(ventas ~ categoria,
                       data = df[df$edad > 30, ],
                       FUN = mean)
```

→ *Sintaxis anidada. Primero se filtra df[df$edad > 30,], luego se aplica aggregate(). Menos legible.*

Tidyverse (dplyr):

```
# Operaciones encadenadas con pipe
resultado <- df |>
  filter(edad > 30) |>
  group_by(categoria) |>
  summarise(ventas_promedio = mean(ventas))
```

→ *Código lineal y legible. Se lee como instrucciones: 'toma df, luego filtra, luego agrupa, luego resume'.*

data.table:

```
# Todo en una expresión concisa
resultado <- DT[edad > 30,
                .(ventas_promedio = mean(ventas)),
                by = categoria]
```

Comparación: Unir (Join) Tablas

Los joins combinan información de dos tablas basándose en columnas comunes. Cada enfoque tiene su sintaxis particular.

R Base - merge():

```
resultado <- merge(df1, df2, by = "id", all.x = TRUE)
```

→ *all.x = TRUE hace left join. all.y = TRUE hace right join. all = TRUE hace full join.*

Tidyverse - left_join():

```
resultado <- df1 |> left_join(df2, by = "id")
```

→ *Funciones específicas: left_join(), right_join(), inner_join(), full_join(). Muy intuitivo.*

data.table - sintaxis de join:

```
resultado <- DT1[DT2, on = "id"]  # right join por defecto
resultado <- DT2[DT1, on = "id"]  # left join (invertir orden)
```

→ *Join por referencia (extremadamente rápido). Orden importa: X[Y] hace right join con Y.*

Comparación: Reestructurar Datos (Wide ↔ Long)

Reorganizar datos entre formato 'ancho' (wide) y 'largo' (long) es común en análisis de datos.

R Base - reshape() [menos usado]:

```
long <- reshape(df, direction = "long",
                varying = c("v1", "v2", "v3"),
                v.names = "valor", timevar = "variable")
```

→ *Sintaxis compleja y poco intuitiva. Genera warnings. Difícil de recordar parámetros.*

Tidyverse - pivot_longer() / pivot_wider():

```
long <- df |> pivot_longer(cols = c(v1, v2, v3),
                           names_to = "variable",
                           values_to = "valor")
```

→ *Sintaxis intuitiva y consistente. names_to y values_to describen claramente el resultado.*

data.table - melt() / dcast():

```
long <- melt(DT, measure.vars = c("v1", "v2", "v3"),
             variable.name = "variable", value.name = "valor")
```

→ *Similar a tidyr pero mucho más rápido. melt() para long, dcast() para wide.*

Guía Práctica: ¿Cuándo Usar Cada Enfoque?

🔧 R Base

Usar cuando...

- Scripts simples y rápidos
- No se pueden instalar paquetes
- Datasets muy pequeños (<10,000 filas)
- Necesitas portabilidad absoluta
- Estás aprendiendo R

Ejemplo:
Análisis ad-hoc de encuesta pequeña

📊 Tidyverse

Usar cuando...

- Análisis exploratorio
- Reportes y comunicación
- La legibilidad es prioritaria
- Datasets pequeños a medianos (<1M filas)
- Trabajas en equipo

Ejemplo:
Informe mensual con visualizaciones

⚡ data.table

Usar cuando...

- Datasets grandes (>1M filas)
- Procesamiento en producción
- Velocidad es crítica
- Memoria es limitada
- ETL y pipelines

Ejemplo:
Procesamiento diario de logs (5GB)

Tabla Comparativa: Resumen Ejecutivo

Criterio	R Base	Tidyverse	data.table
Velocidad de procesamiento	⭐⭐	⭐⭐⭐⭐	⭐⭐⭐⭐⭐
Legibilidad del código	⭐⭐⭐	⭐⭐⭐⭐⭐	⭐⭐⭐
Curva de aprendizaje	⭐⭐⭐⭐	⭐⭐⭐⭐	⭐⭐
Eficiencia de memoria	⭐⭐	⭐⭐⭐	⭐⭐⭐⭐⭐
Datasets grandes (>1M)	⭐	⭐⭐⭐	⭐⭐⭐⭐⭐
Comunidad y documentación	⭐⭐⭐⭐⭐	⭐⭐⭐⭐⭐	⭐⭐⭐⭐
Velocidad de importación	⭐⭐	⭐⭐⭐⭐	⭐⭐⭐⭐⭐
Compatibilidad	⭐⭐⭐⭐⭐	⭐⭐⭐⭐	⭐⭐⭐⭐

Caso Práctico: Pipeline Completo de Análisis

Objetivo: Analizar ventas del último año, calcular ganancias por región y trimestre, identificar top performers

📊 Tidyverse (Legible y expresivo):

```
library(tidyverse)

# Pipeline completo y legible
datos <- read_csv("ventas.csv")

resultado <- datos |>
  # 1. Filtrar último año
  filter(fecha >= "2024-01-01") |>
  # 2. Calcular métricas
  mutate(
    trimestre = quarter(fecha),
    ganancia = precio - costo,
    margen_pct = (ganancia / precio) * 100
  ) |>
  # 3. Agrupar y resumir
  group_by(region, trimestre) |>
  summarise(
    ventas_total = sum(ventas),
    ganancia_total = sum(ganancia),
    margen_promedio = mean(margen_pct),
    transacciones = n(),
    .groups = "drop"
  ) |>
  # 4. Ordenar resultados
  arrange(desc(ganancia_total))
```

⚡ data.table (Conciso y rápido):

```
library(data.table)

# Todo en expresiones concisas
datos <- fread("ventas.csv")

# 1-2. Filtrar y calcular en un paso
datos[fecha >= "2024-01-01",
  `:=`(ganancia = precio - costo,
       margen_pct = (precio - costo) / precio * 100)]

# 3-4. Agrupar, resumir y ordenar
resultado <- datos[,
  .(ventas_total = sum(ventas),
    ganancia_total = sum(ganancia),
    margen_promedio = mean(margen_pct),
    transacciones = .N),
  by = .(region,
         trimestre = quarter(fecha))
][order(-ganancia_total)]

# Resultado idéntico, ~5x más rápido
```

Buena Práctica: Combinar Enfoques

💡 Los tres paradigmas pueden coexistir en un mismo proyecto:

Flujo de trabajo típico en un proyecto real:

1. Importación con data.table:

```
DT <- fread('datos_grandes.csv') # Rápido
```

2. Exploración con Tidyverse:

```
DT |> as_tibble() |> glimpse()  # Convertir para explorar
```

3. Procesamiento pesado con data.table:

```
DT[, nuevas_vars := calculos]  # Eficiente en memoria
```

4. Visualización con ggplot2 (tidyverse):

```
ggplot(DT, aes(x, y)) + geom_point()  # data.table compatible
```

📌 Consejo: Elige la herramienta apropiada para cada tarea

Recursos para Profundizar

📖 Documentación oficial:

• R Base: ?read.csv, ?merge, ?aggregate en consola de R

• Tidyverse: https://www.tidyverse.org

• data.table: https://rdatatable.gitlab.io/data.table

📚 Libros recomendados:

• 'R for Data Science' (Wickham & Grolemund) - Tidyverse

Disponible gratis: https://r4ds.hadley.nz

• 'data.table in R' - Tutorial oficial

vignettes: vignette('datatable-intro')

🎯 Práctica recomendada:

• Reproducir análisis propios con los tres enfoques

• Comparar código y tiempos de ejecución reales

• Participar en comunidades: Stack Overflow, R-bloggers

Consideraciones para el Trabajo Final del Curso

🎓 Sugerencias según el tipo de proyecto:

Proyecto con datos pequeños/medianos (<500K filas):

✓ Usar Tidyverse

✓ Enfocarse en legibilidad y reproducibilidad

✓ Integrar visualizaciones con ggplot2

✓ Documentar bien el proceso (RMarkdown/Quarto)

Proyecto con datos grandes (>1M filas):

✓ Usar data.table para procesamiento

✓ Documentar decisiones de optimización

✓ Incluir benchmarks comparativos

✓ Considerar escalabilidad y memoria

¡Gracias!

Preguntas y Discusión

Aplicaciones de Ciencia de Datos | UNO 2025